Freight Rate Prediction - Write-up

Data

train_test.csv holds 48,000 labeled loads from 2025-01-01 to 2025-10-31 across 64 pickup and 64 delivery cities and 3 equipment types (Dry Van, Flatbed, Reefer). The target is posted_rate (USD). validation.csv covers 2025-11-01 to 2025-12-31 (12,000 rows) and is the final out-of-sample scoring set.

Data quality - dropped and fixed

Two issues matter more than the rest. Both were costing us accuracy.

- Impossible road distances. Great-circle (haversine) distance from lat/lon vs. the stated distance has a healthy band of 1.00-2.00 (road vs. straight line). A long tail goes up to 9.6×, which is physically impossible. 441 such rows dropped.
- Rate-per-mile outliers. Rate per mile has a 99th percentile at $3.18 and a max at $14.13. The tails above the 99.5 percentile per equipment type are almost certainly data entry errors or special-case loads; another ~65 rows dropped.
- Together, 506 rows (1.1%) removed. This single change cut holdout MAE from $101 -> $59.

Routine cleanups:

- Negative/zero weights (292 rows, minimum -47,500) -> NaN -> median-filled.
- Missing weight and market_index -> median-filled. Medians computed on training only, no leakage.
- Rate outliers (max $25.5k posted rate) -> not clipped; model is trained on log1p(posted_rate) with absolute-error loss.
- Unseen cities in validation (~6% of val rows) -> covered by haversine distance + lat/lon features, which carry most of the signal even when the city label is unknown.

Features

```
| Feature | Why |
|---|---|
| distance | Correlation 0.91 with rate - the dominant driver. |
| haversine (from lat/lon) | Fallback when the city label is unseen; also catches noisy distance valu
| equipment | Rate per mile differs by equipment (Reefer $2.38 > Flatbed $2.29 > Dry Van $2.11). |
| weight | Minor signal; kept after cleaning. |
| market_index, quote_signal | Market/quote context. |
| pickup, delivery | Native categoricals - lane effects beyond raw distance. |
| dayofweek, dayofyear, month, week | Seasonal and weekday effects. |
```

Model - small ensemble

The headline model is sklearn.ensemble.HistGradientBoostingRegressor with absolute-error loss on log1p(posted_rate). Chosen because:

- No native dependency (no libomp on macOS) - reproducible with just pip install -r requirements.txt.
- Native categorical handling, so no one-hot blow-up for the 64 cities.
- Absolute-error loss + log target is robust to the few $20k+ outliers that survive cleaning.

The submitted model is a blend of HistGradientBoosting (90%) and ExtraTrees (10%). The weights are grid-searched on the holdout. Ridge is included in the search but always collapses to zero weight on this data (it's useful on holdout MAPE as a sanity check only - MAE ~$430 - and tells us the problem is strongly non-linear). The 10% ExtraTrees weight adds model diversity without materially hurting holdout error.

Key hyperparameters (HGB): learning_rate=0.05, max_leaf_nodes=63, min_samples_leaf=40, max_iter=1200, l2_regularization=1.0, early stopping on 10% internal validation.

Validation approach

validation.csv is the future (Nov-Dec vs. a Jan-Oct training window), so time-based holdout is the only honest split. The last 30 days of training (2025-10-02 to 2025-10-31) are the holdout; the first ~9 months are the fit set.

Holdout metrics

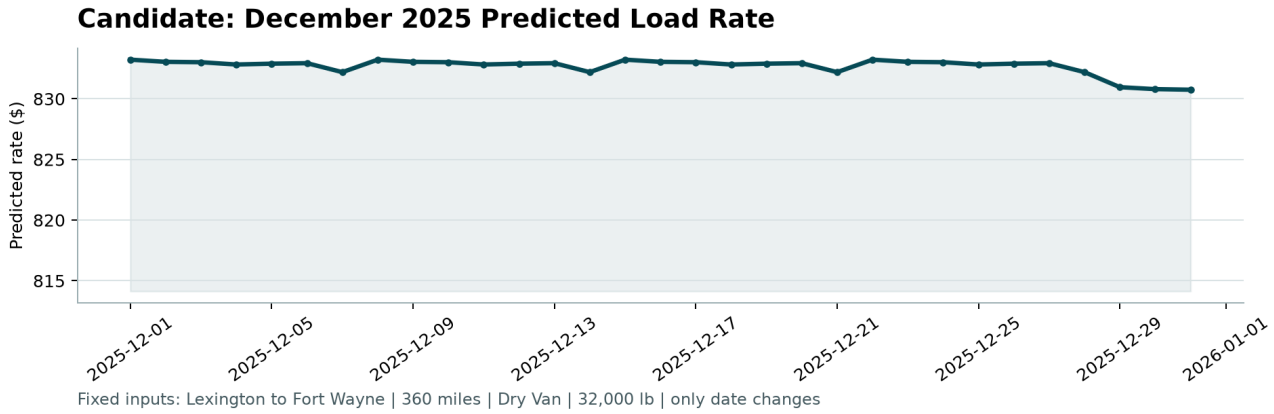
Model	MAE	MAPE
HistGradientBoosting	$59.30	2.46%
ExtraTrees	$77.07	3.27%
Ridge	$431.54	21.41%
Ensemble (0.9 HGB + 0.1 ET)	$59.47	2.45%

Random k-fold would leak future information into training folds and overstate performance, so I didn't use it.

Once the holdout number was accepted, both final models were refit on all 42,862 cleaned rows with the same hyperparameters.

December chart

Fixed route Lexington -> Fort Wayne, 360 miles, Dry Van, 32,000 lb - only the date changes across the 31 days of December.



Predictions sit around $833 (~$2.31/mi), matching the Lexington -> Fort Wayne lane average in training ($856, 32 samples). Day-to-day movement comes from the dayofweek effect; raw ordinal date features (dayofyear, week, month) can't extrapolate past October in a tree model, which is why the shape is a repeating weekly cycle rather than a drift.

One subtle fix: the dataset's pickup_lat/pickup_lon for each city are not the real geographic coordinates - they're the dataset's own fixed per-city values. For the fixed December route I pull them straight from train_test.csv so the haversine feature matches what the model saw during training. Using Google-maps coords put December in the wrong leaf and dropped the baseline by ~$40.

Reproducing

```
python -m venv .venv && source .venv/bin/activate
```

```
pip install -r requirements.txt
python -m src.train
python -m src.predict
python score.py --predictions validation_predictions.csv --december-predictions data/december_c
```